FreelanceHub

MERN Stack Freelancing Platform

Project Documentation

Technology	MERN-Stack
Academic Year	2026–2027

Prepared By

Mansi Wagh

Introduction

Welcome to FreelanceHub — a freelancing platform built to make hiring and finding freelance work simple, secure, and fast. From posting a project to finding the right freelancer, approving an application, and discussing the work in real time, FreelanceHub covers the complete hiring journey end to end.

Clients can post projects with a clear title, description, budget, duration, and the skills they need. Freelancers can browse every open project, filter them to match their own skills, and apply with a short proposal, their rate, and a link to their portfolio — so clients can judge their work before hiring.

With a clean, role-based interface, each user sees a dashboard built for them. Clients review the applications they receive and approve or reject each one, while freelancers track every application they've sent as it moves from "Pending" to "Approved" or "Rejected."

Once a client approves an application, the two can talk instantly through a built-in real-time chat powered by Socket.io — no emails, no switching apps. Behind the scenes, JWT authentication, bcrypt password hashing, and role-based route protection make sure every user only sees and does what their role allows.

Scenario Based Case Study

Background

"FreelanceHub" is a full-stack web application designed to connect clients who need work done with skilled freelancers who want to do it. The platform combines simple project posting, transparent applications, and real-time communication in one place, with secure role-based access for clients, freelancers, and admins.

Scenario

User Persona:

- **Name:** Sarah Lopez
- **Age:** 22
- **Profession:** Recent graphic design graduate starting her freelance career
- **Goal:** Find genuine design projects that match her skills and build a strong portfolio
- **Challenges:** No industry contacts, difficulty finding real clients, and slow back-and-forth over email before any work begins

The Challenge

Sarah had tried posting her work on social media and messaging small businesses directly, but most never replied, and the few that did took days to respond over email. She needed a platform that could:

- Show her real, open projects that match her skills in one place
- Let her apply quickly with a proposal, her rate, and her portfolio
- Show her clearly whether each application is pending, approved, or rejected
- Let her talk to the client directly once she's hired

Solution: How FreelanceHub Helped

When Sarah joined FreelanceHub as a freelancer, she filtered the open projects by her skills and found a logo redesign posted by a local bakery, "Sugar Rush." The platform gave her:

- **Relevant projects:** Filtering showed her only the projects that matched her design skills and budget expectations.
- **A quick application:** A single form to send her proposal, rate, and portfolio link, saved instantly with a "Pending" status.
- **Clear status tracking:** Her dashboard showed the moment the client approved her application.
- **Instant communication:** As soon as she was approved, a real-time chat opened with the client, so they could discuss colours, fonts, and drafts right away.

Within a day, Sarah was hired, shared her first drafts in the chat, and started building her freelance portfolio with a real client.

System Requirements

To ensure smooth development, deployment, and usage of the FreelanceHub web application, certain system prerequisites must be met. These requirements are categorized into software and hardware specifications.

1. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux — supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for the frontend build tooling and powers the Express backend.
- **npm (v8 or above):** Package manager used to install all frontend and backend dependencies.
- **React.js + Vite:** React is the UI library; Vite is the build tool and dev server used to scaffold and run the client.
- **Bootstrap:** CSS framework used for responsive layouts and styling.
- **React Router DOM:** Handles client-side routing across all pages.
- **React Toastify:** Shows success and error notifications to the user.
- **Axios:** HTTP client used for all communication between the frontend and backend.
- **Express.js:** Lightweight Node.js web framework used to build the REST API.
- **Mongoose:** ODM (Object Data Modeling) library used to define schemas and interact with MongoDB.
- **MongoDB Atlas:** Cloud-hosted NoSQL database used to store all application data (users, freelancer profiles, projects, applications, chats).
- **JWT (jsonwebtoken) & bcryptjs:** Used for authentication — signing session tokens and hashing passwords.
- **Socket.io & socket.io-client:** Used for real-time, two-way chat between clients and freelancers.
- **Browser:** Google Chrome / Firefox (latest version) — for rendering and testing the UI in real time.
- **Postman:** Used for testing API endpoints during backend development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.
- **Git & GitHub:** Used for version control and hosting the source code.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — for running development servers, the code editor, and browser testing simultaneously.

- **Storage:** At least 1 GB free space — required for package installations and local project files.
- **Display:** 1366x768 or higher — recommended for a comfortable coding and layout-review experience.
- **Internet Connection:** Required for connecting to MongoDB Atlas and installing packages.

Technical Architecture

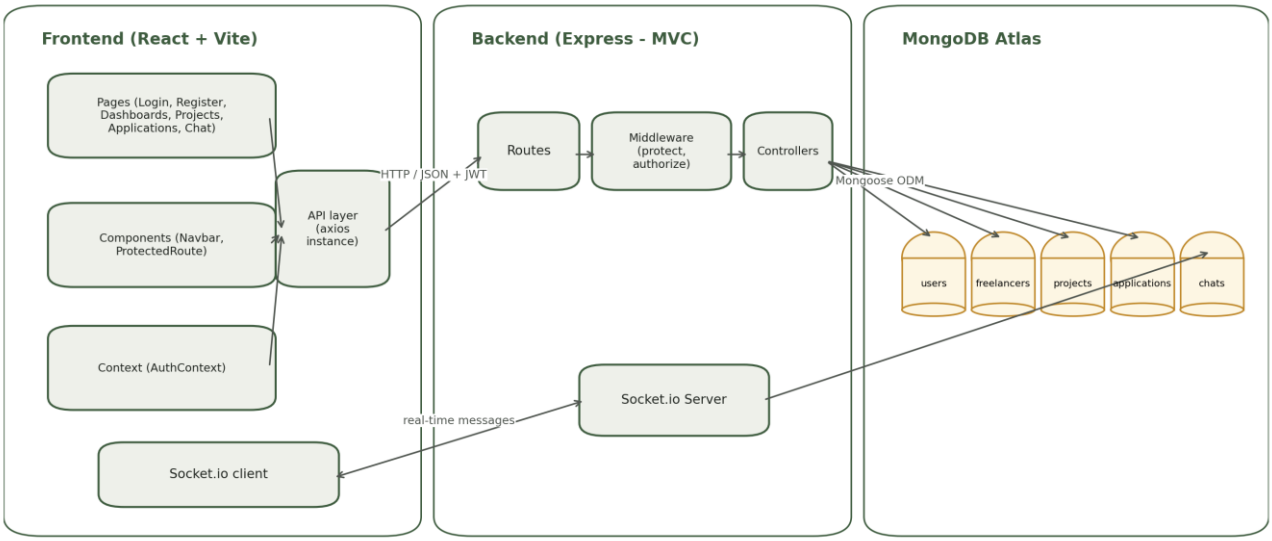


Figure 1: FreelanceHub system architecture

Technical Architecture Overview

FreelanceHub is built on a three-layer architecture that cleanly separates presentation, business logic, and data storage — a standard MERN-stack design, extended with Socket.io for real-time chat, that keeps the codebase modular and easy to extend.

Frontend (React + Vite): A single-page application composed of Pages (Login, Register, the client and freelancer dashboards, project and application pages, and Chat), reusable Components (Navbar, ProtectedRoute), and a global AuthContext that tracks the logged-in user and their role across the app. All REST calls flow through a single shared axios instance.

API layer / Axios instance: A configured axios client that automatically attaches the user's JWT to every outgoing request, so individual pages never have to handle authentication headers manually.

Backend — Routes (View layer): Express route files define every REST endpoint, grouped by resource: /api/auth, /api/projects, and /api/applications.

Middleware: Two middleware functions gate access — protect verifies the JWT and attaches the logged-in user to the request, and authorize restricts a route to specific roles (for example, only clients can post projects and only freelancers can apply).

Controllers: Contain all business logic — validating input, saving projects, recording applications, updating application status, and opening a chat when an application is approved.

Real-time Layer — Socket.io: A Socket.io server runs alongside Express. Approved clients and freelancers join a shared chat room, and every message is saved to the database and pushed instantly to the other person.

Database — MongoDB Atlas: A cloud-hosted MongoDB cluster storing five collections: users, freelancers, projects, applications, and chats, accessed through Mongoose schemas that enforce structure and validation.

ER Diagram

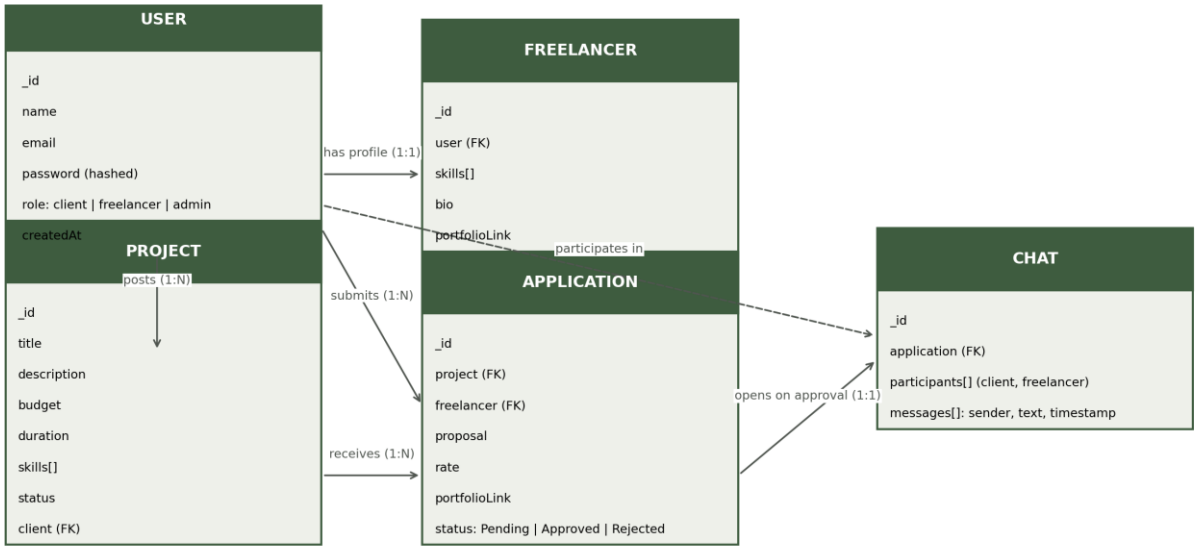


Figure 2: FreelanceHub entity-relationship diagram

Entities and Relationships

1. User

- Represents every person on the platform — clients, freelancers, and administrators.
- Each user has a unique profile with name, email, a securely hashed password, and a role ("client", "freelancer", or "admin") that decides which pages and actions they can access.

Relationships:

- Posts → Projects: A client can post many projects.
- Has → Freelancer Profile: Each freelancer has exactly one profile.
- Submits → Applications: A freelancer can apply to many projects.
- Participates in → Chats: A user can take part in many chats.

2. Freelancer

- Stores extra profile details for users with the freelancer role, such as skills, a short bio, and a portfolio link.
- Linked to exactly one User.

3. Project

- Represents a piece of work posted by a client, with a title, description, budget, duration, required skills, and a status.

Relationships:

  - Posted by → User: Each project belongs to the client who created it.
  - Receives → Applications: A project can receive many applications from different freelancers.

4. Application

- Created when a freelancer applies to a project. Stores the proposal, the freelancer's rate, a portfolio link, and a status that moves from "Pending" to "Approved" or "Rejected."
- Links exactly one freelancer to exactly one project.

5. Chat

- Created when a client approves an application, connecting that client and freelancer.
- Stores the two participants and an array of messages, each with the sender, the text, and a timestamp, so the full conversation history is kept.

Key Features

1. User Management

- **Registration & Authentication:** Users create an account and log in securely; passwords are hashed with bcrypt and sessions are managed with signed JWTs.
- **Role-Based Access:** A single role field separates clients, freelancers, and admins, and protected routes on both the frontend and backend enforce what each role can do.

2. Project Management

- **Post Projects:** Clients post projects with a title, description, budget, duration, and required skills.
- **My Projects:** Clients see and manage every project they have posted from their dashboard.

3. Project Browsing & Applications

- **Browse & Filter:** Freelancers see all open projects and filter them by skills and budget.
- **Apply with Proposal:** Freelancers apply with a proposal, their rate, and a portfolio link.

4. Application Management

- **Review Applications:** Clients see every application received for each of their projects.
- **Approve or Reject:** Clients approve the best fit and reject the rest with one click.
- **Status Tracking:** Freelancers track each application as Pending, Approved, or Rejected.

5. Real-time Chat

- **Instant Messaging:** Once an application is approved, the client and freelancer chat in real time through Socket.io, with no page refresh.
- **Saved History:** Every message is stored in MongoDB, so the conversation is still there the next time they log in.

6. Notifications & User Experience

- **Toast Notifications:** React Toastify shows clear success and error messages after every action.
- **Responsive Design:** Bootstrap keeps every page usable on desktop, tablet, and mobile.

Roles and Responsibilities

Client

- **Registration:** Clients create an account by providing their name, email, password, and choosing the Client role.
- **Project Posting:** Clients post new projects with all the details freelancers need to apply.
- **Project Management:** Clients view and manage all the projects they have posted.
- **Reviewing Applications:** Clients compare proposals, rates, and portfolios for each project.
- **Hiring Decisions:** Clients approve or reject each application.
- **Communication:** Clients chat in real time with the freelancers they approve.
- **Logout:** Clients can securely log out, clearing their session.

Freelancer

- **Registration:** Freelancers create an account by providing their name, email, password, and choosing the Freelancer role.
- **Project Exploration:** Freelancers browse and filter open projects to find work that matches their skills.
- **Applying:** Freelancers submit applications with a proposal, rate, and portfolio link.
- **Tracking Applications:** Freelancers follow the status of every application they've sent.
- **Communication:** Freelancers chat in real time with clients who approve their application.
- **Logout:** Freelancers can securely log out, clearing their session.

Admin

- **Platform Oversight:** The admin role is built into the authentication system, giving a protected level of access above clients and freelancers.
- **Future Responsibilities:** The role is designed to be extended with user verification, content moderation, and dispute handling.

User Flow

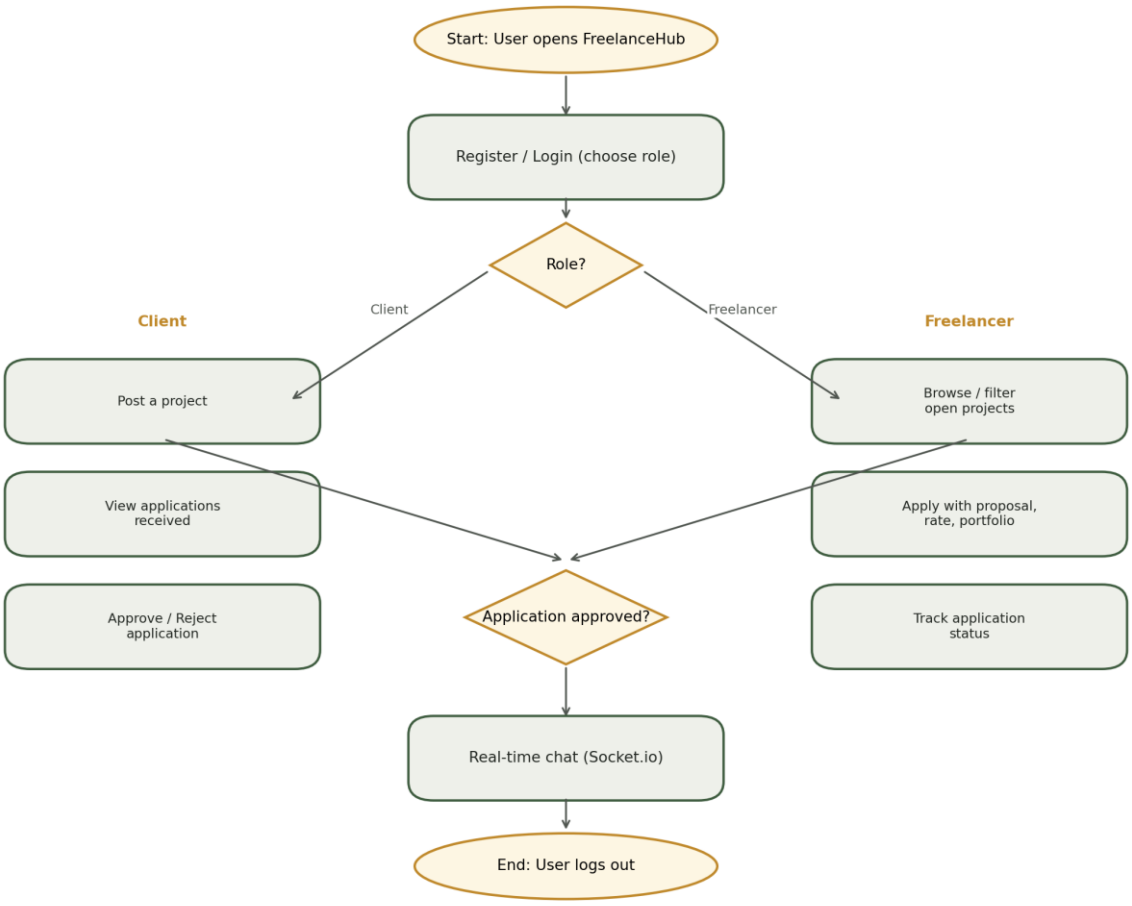


Figure 3: FreelanceHub end-to-end user flow

Start: The user opens the FreelanceHub application.

Register / Login: The user creates an account and chooses the Client or Freelancer role, or logs in with an existing account and receives a JWT.

Role-based Dashboard: The user is taken to the dashboard for their role.

Client — Post a Project: The client fills in the project title, description, budget, duration, and required skills.

Client — View Applications: The client opens a project to see every application received.

Client — Approve / Reject: The client approves the best application and rejects the others.

Freelancer — Browse Projects: The freelancer browses open projects and filters them by skills and budget.

Freelancer — Apply: The freelancer submits a proposal, rate, and portfolio link; the application is saved as Pending.

Freelancer — Track Status: The freelancer checks whether each application is Pending, Approved, or Rejected.

Chat: Once an application is approved, the client and freelancer chat in real time to discuss the work.

MVC Pattern

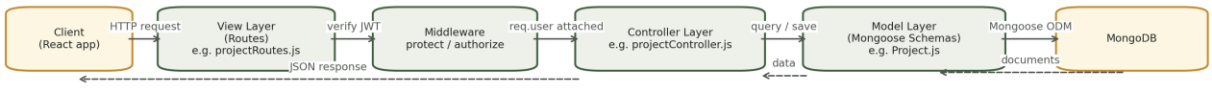


Figure 4: Request flow through FreelanceHub's MVC layers

The FreelanceHub backend follows the Model-View-Controller (MVC) architectural pattern, separating the application into three interconnected layers for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer handles all data-related logic, implemented with Mongoose schemas in the Server/models folder — User.js, Freelancer.js, Project.js, Application.js, and Chat.js. Each schema defines its fields, data types, required values, allowed values (such as the role and status lists), and references to other collections.

Controller Layer

The Controller layer, in Server/controllers, acts as the intermediary between routes and models. Each controller function receives the incoming request, validates input, performs the necessary business logic (such as hashing a password at registration or changing an application's status), calls the appropriate model methods, and returns a response.

View Layer (Routing Layer)

In this REST API context, the "View" is implemented as the routing layer in Server/Routes. Each route file defines the endpoints for one resource (for example, the project routes define POST /api/projects, GET /api/projects, and GET /api/projects/my-projects) and wires each endpoint to its controller function — passing it through the protect and authorize middleware first where needed.

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features (like chat) were added by creating a new model, controller, and route, without changing existing ones.
- **Reusability:** Middleware like protect and authorize is reused across every protected route in the app.
- **Testability:** Each controller can be tested independently of the routing layer.
- **Collaboration-Friendly:** Different developers can work on different resources (projects, applications, chats) at the same time without conflicts.

Project Setup and Configuration

Creating the Project Folder

- Create a new folder named FreelanceHub.
- Inside it, create two folders: Client and Server.
- Open the FreelanceHub folder in Visual Studio Code.

Client Setup (React + Vite)

```
cd FreelanceHub
npm create vite@latest Client -- --template react
cd Client
npm install
npm install react-router-dom axios react-toastify bootstrap socket.io-client
```

This scaffolds a React app with Vite, then adds React Router for page navigation, Axios for API calls, React Toastify for notifications, Bootstrap for styling, and the Socket.io client for real-time chat.

Server Setup (Express + MongoDB)

```
cd FreelanceHub
mkdir Server && cd Server
npm init -y
npm install express mongoose cors dotenv bcryptjs jsonwebtoken socket.io
npm install -D nodemon
```

After installing, add the dev and start scripts to package.json:

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

Then create a .env file inside the Server folder:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
```

Create Database Connection

The server connects to MongoDB Atlas before it starts handling requests:

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`MongoDB connection error: ${error.message}`);
    process.exit(1);
  }
};
```

```
module.exports = connectDB;
```

Create Schemas & Models

Using the entities from the ER Diagram, each collection is defined as a Mongoose schema. Three representative examples:

User.js — defines the schema for user accounts and their role:

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ["client", "freelancer", "admin"], default: "freelancer" },
}, { timestamps: true });
```

Project.js — defines the schema for projects posted by clients:

```
const projectSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String, required: true },
  budget: { type: Number, required: true },
  duration: { type: String, required: true },
  skills: [String],
  status: { type: String, default: "open" },
  client: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
}, { timestamps: true });
```

Application.js — defines the schema for freelancer applications:

```
const applicationSchema = new mongoose.Schema({
  project: { type: mongoose.Schema.Types.ObjectId, ref: "Project", required: true },
  freelancer: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  proposal: { type: String, required: true },
  rate: { type: Number, required: true },
  portfolioLink: { type: String },
  status: { type: String, enum: ["Pending", "Approved", "Rejected"], default: "Pending"
},
}, { timestamps: true });
```

API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login and receive JWT token
GET	/api/projects	Get all open projects
POST	/api/projects	Create a new project (client only)
GET	/api/projects/my-projects	Get client's own posted projects
POST	/api/applications	Submit an application (freelancer only)
GET	/api/applications/:projectId	Get applications for a project

POST | /api/auth/register | Register a new user

POST | /api/auth/login | Login and receive JWT token

GET | /api/projects | Get all open projects

POST | /api/projects | Create a new project (client only)

GET | /api/projects/my-projects | Get client's own posted projects

POST | /api/applications | Submit an application (freelancer only)

GET | /api/applications/:projectId | Get applications for a project

Front-End Development

Frontend Structure

```
Client/src/
|-- api/
|   `-- axios.js            (shared axios instance + JWT interceptor)
|-- context/
|   `-- AuthContext.jsx     (global auth state: user, role, login, register, logout)
|-- components/
|   |-- Navbar.jsx
|   `-- ProtectedRoute.jsx
|-- pages/
|   |-- Login.jsx, Register.jsx
|   |-- Dashboard pages (client and freelancer)
|   |-- Project pages (post project, browse projects, my projects)
|   |-- Application pages (apply, view applications, my applications)
|   `-- Chat.jsx
|-- App.jsx                 (route definitions)
`-- main.jsx                (entry point; wraps App in Router + AuthProvider)
```

Folder & File Explanations

- **api/axios.js:** A single configured Axios instance that automatically attaches the logged-in user's JWT to every request via an interceptor.
- **context/AuthContext.jsx:** Provides global authentication state — the current user, their role, and login/register/logout functions — accessible from any component.
- **components/:** Reusable UI pieces used across pages — the Navbar (showing different links for clients and freelancers) and ProtectedRoute (a route guard for logged-in-only and role-specific pages).
- **pages/:** One component per route, covering the full client journey (post → manage → review applications → chat) and the full freelancer journey (browse → apply → track status → chat).
- **App.jsx:** Defines every route using React Router, wrapping protected and role-specific routes in ProtectedRoute.
- **main.jsx:** The entry point that loads Bootstrap and React Toastify styles and wraps the app in the Router and AuthProvider.

Output Screenshots

Application screenshots supplied for this project are inserted below. Screenshots not supplied (toast notification, MongoDB Atlas collections, and Postman testing) remain pending.

Register Page

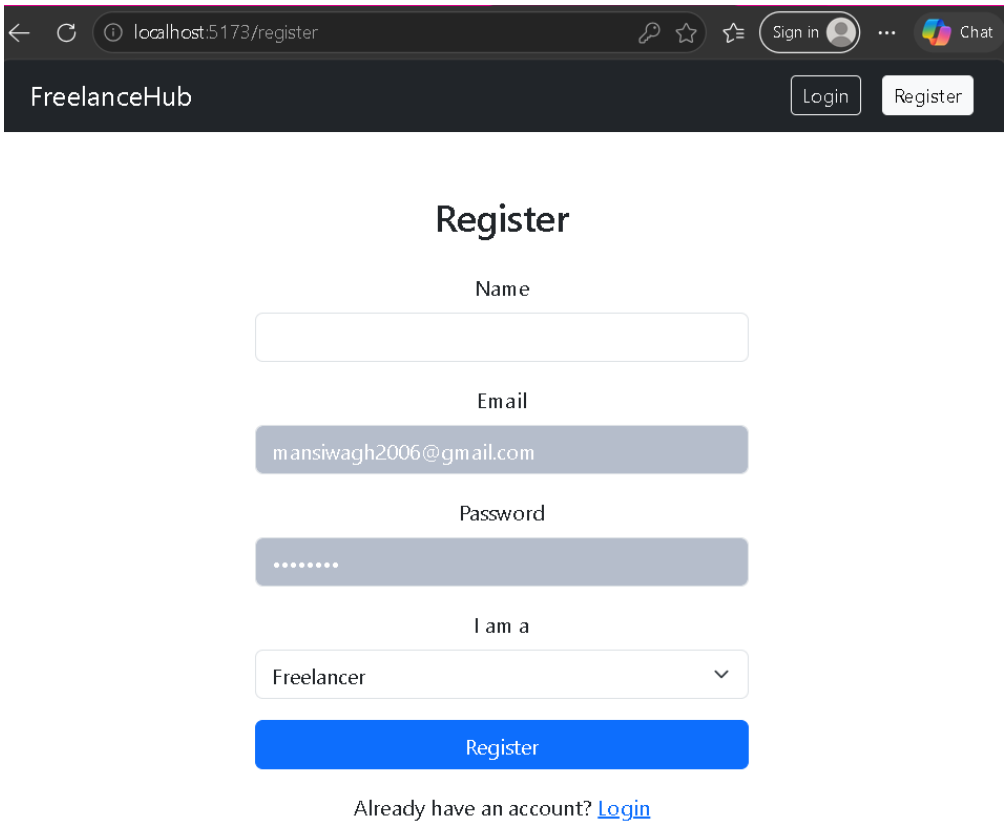


Figure: Register Page

INSERT SCREENSHOT: Register Page — account creation form showing name, email, password and role selection (Client / Freelancer)

Login Page

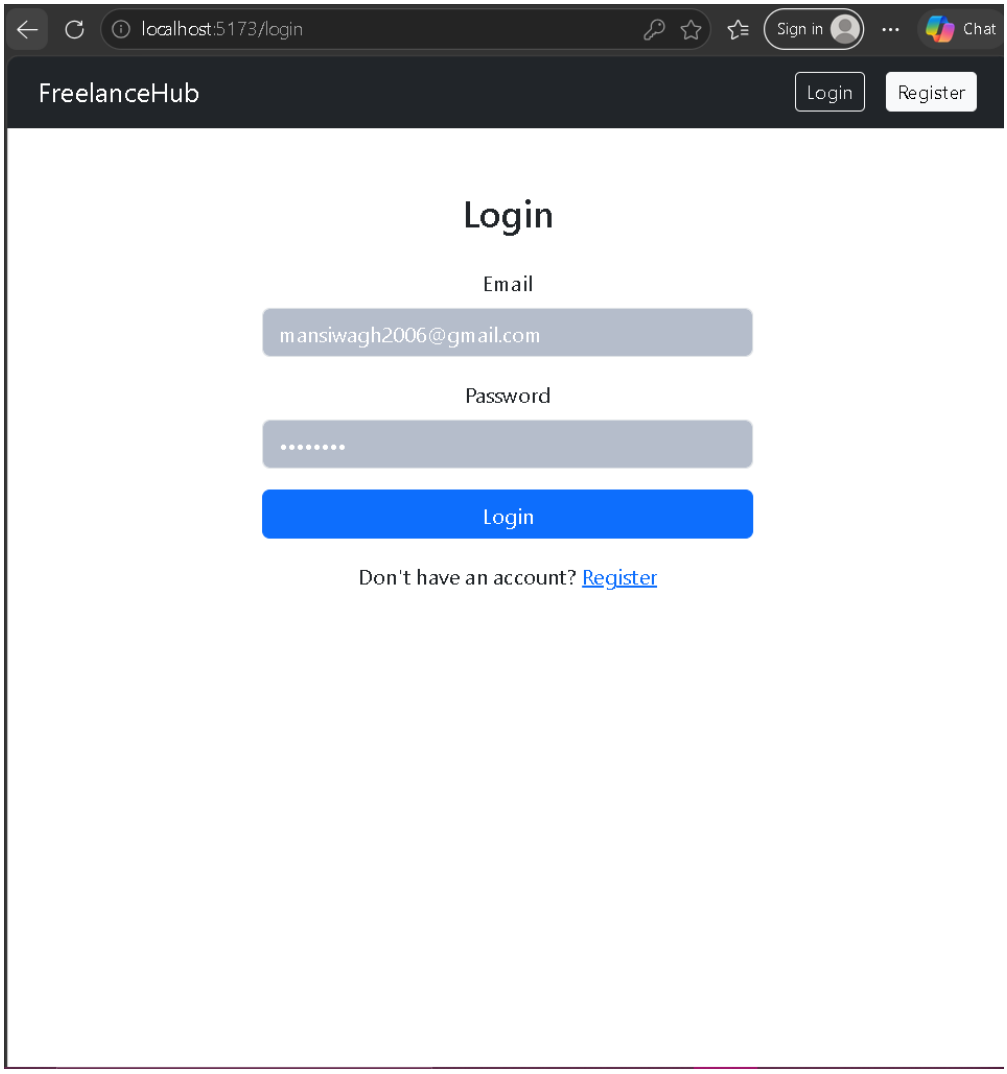


Figure: Login Page

Client Dashboard

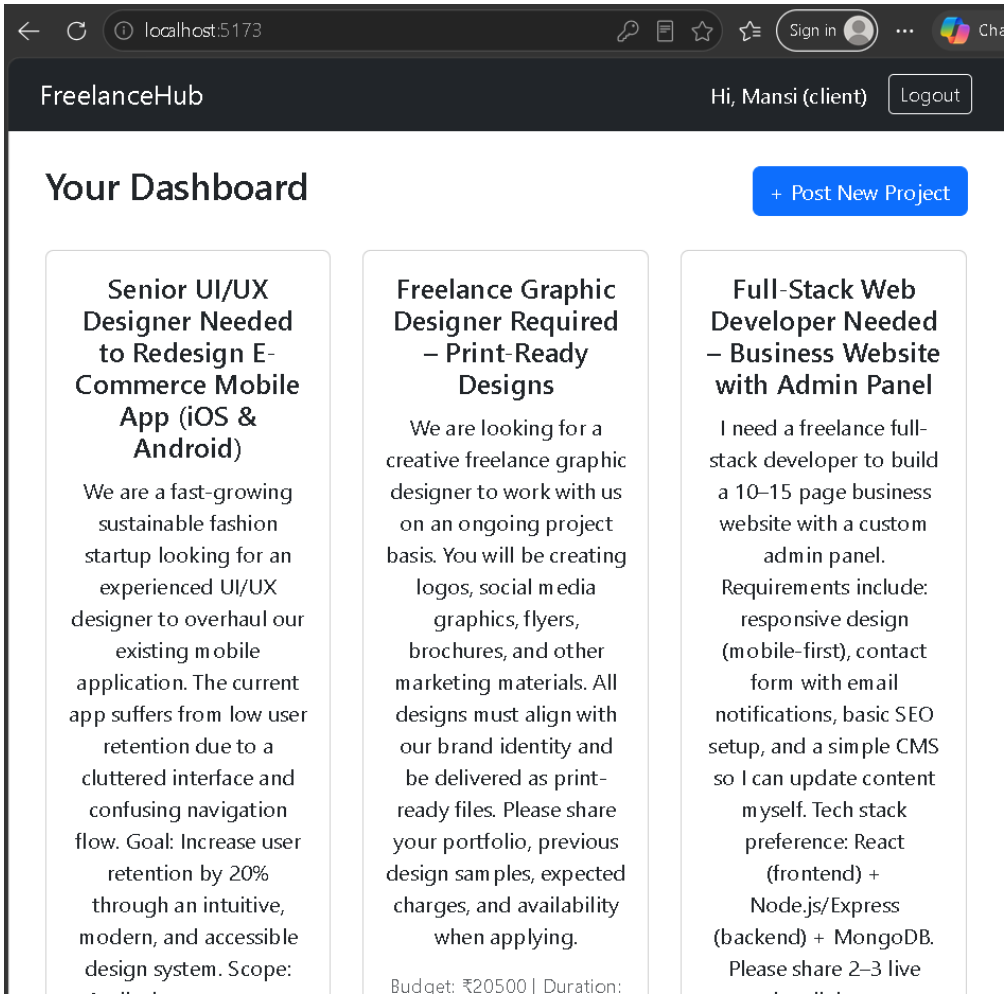


Figure: Client Dashboard

Post a Project

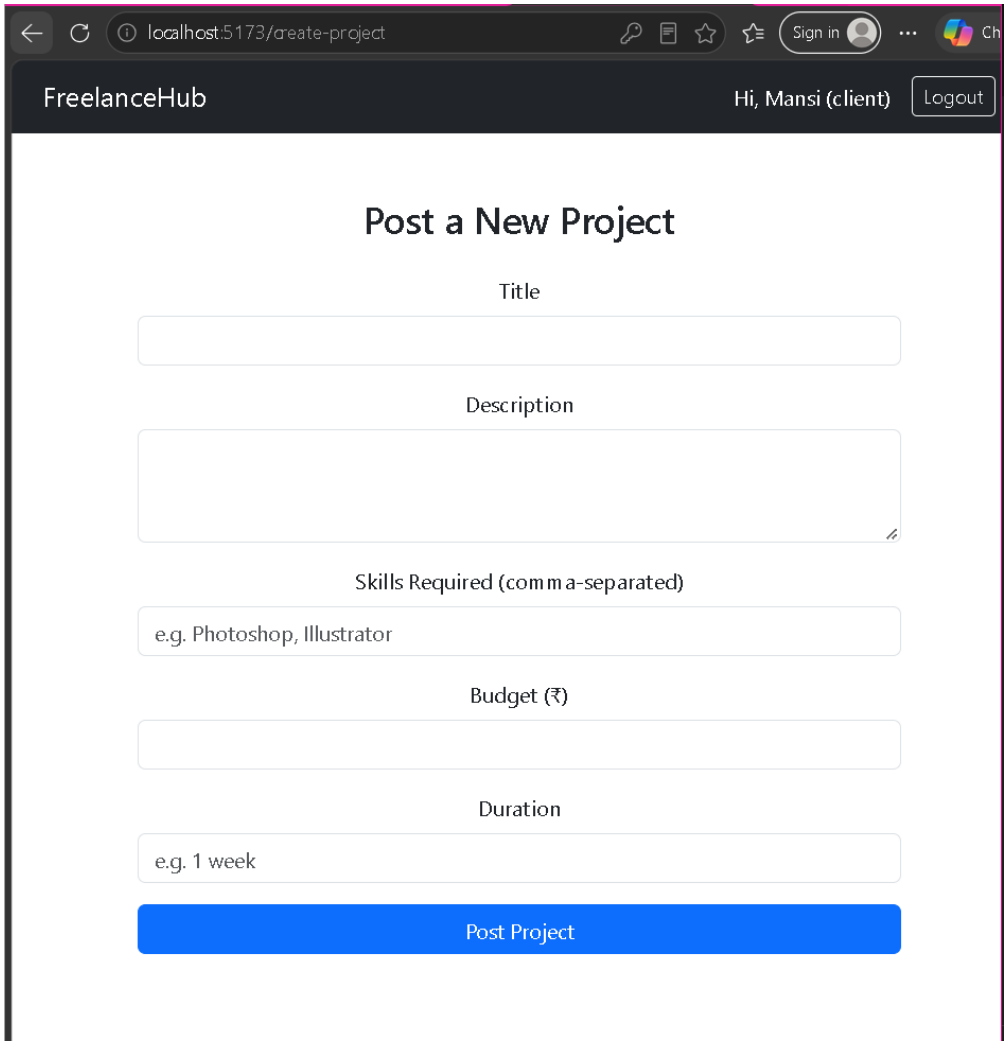


Figure: Post a Project

View Applications (Client)

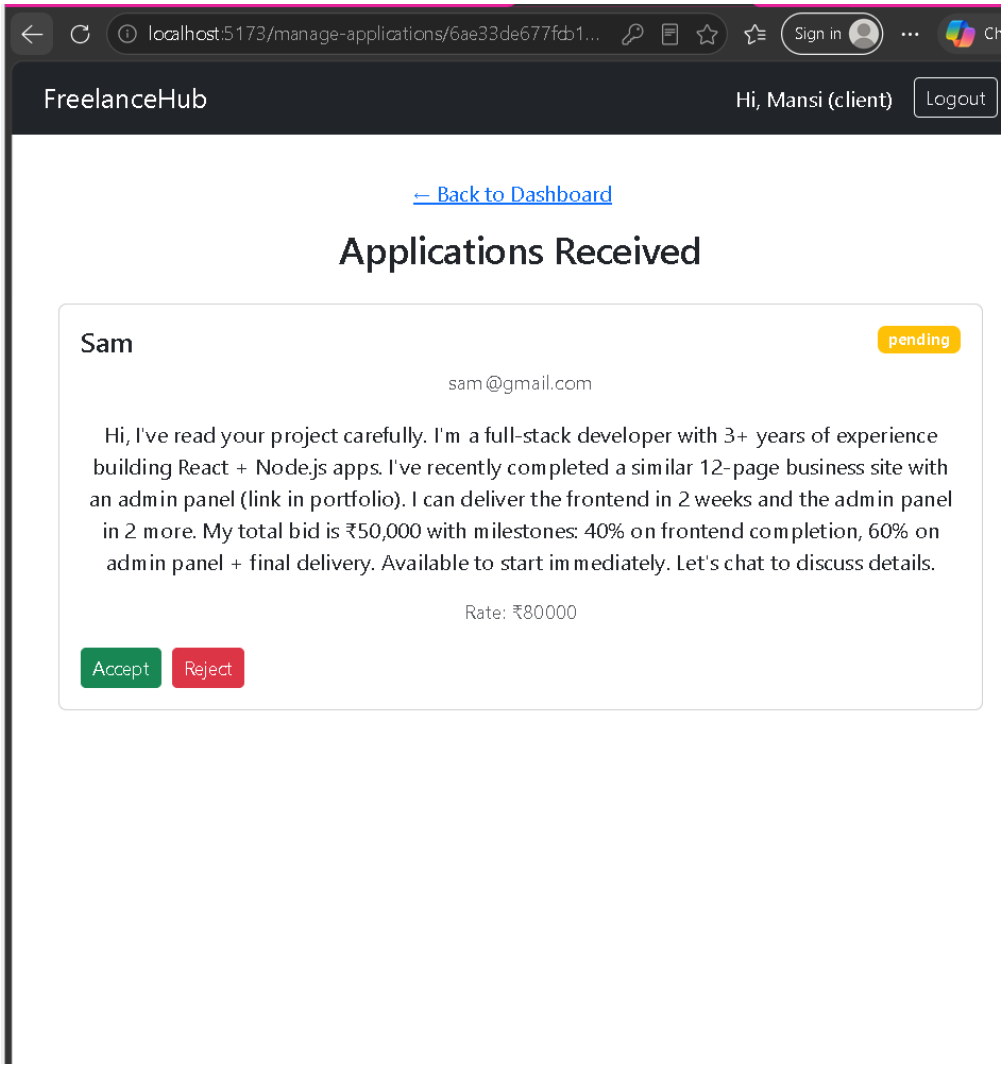


Figure: View Applications (Client)

Freelancer Dashboard / Browse Projects

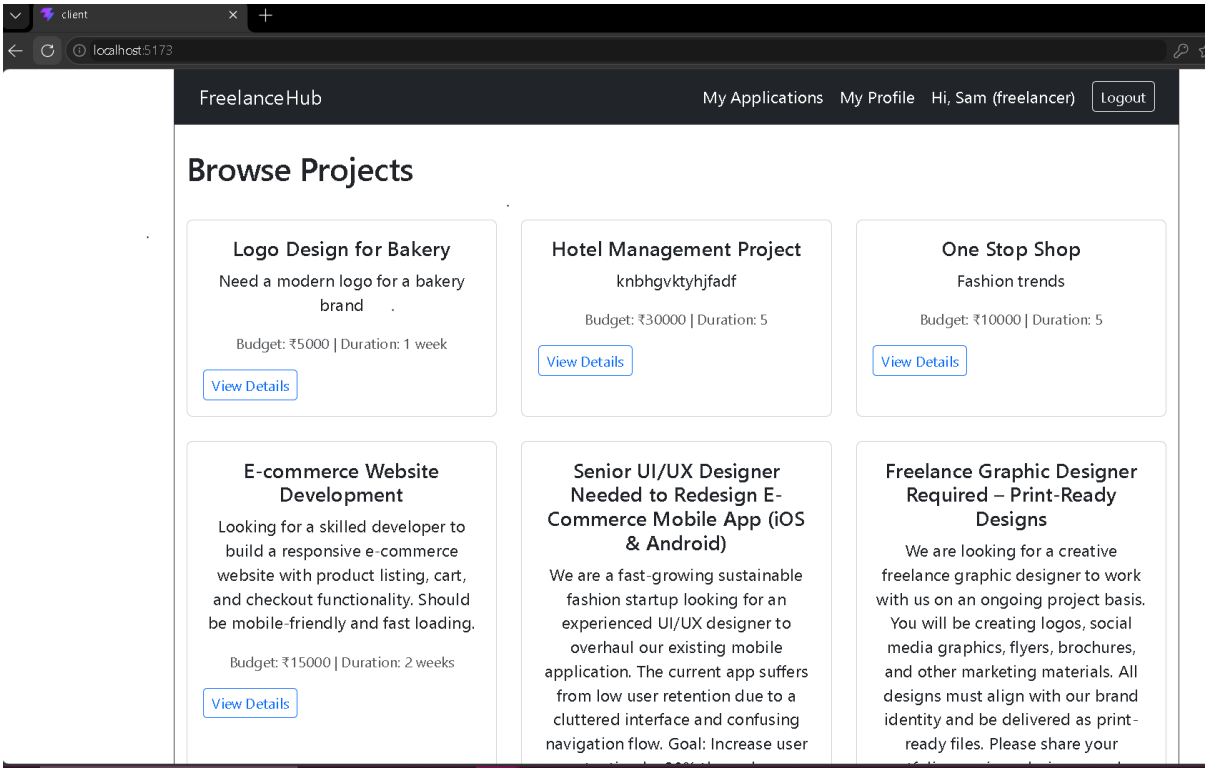


Figure: Freelancer Dashboard / Browse Projects

Apply to a Project

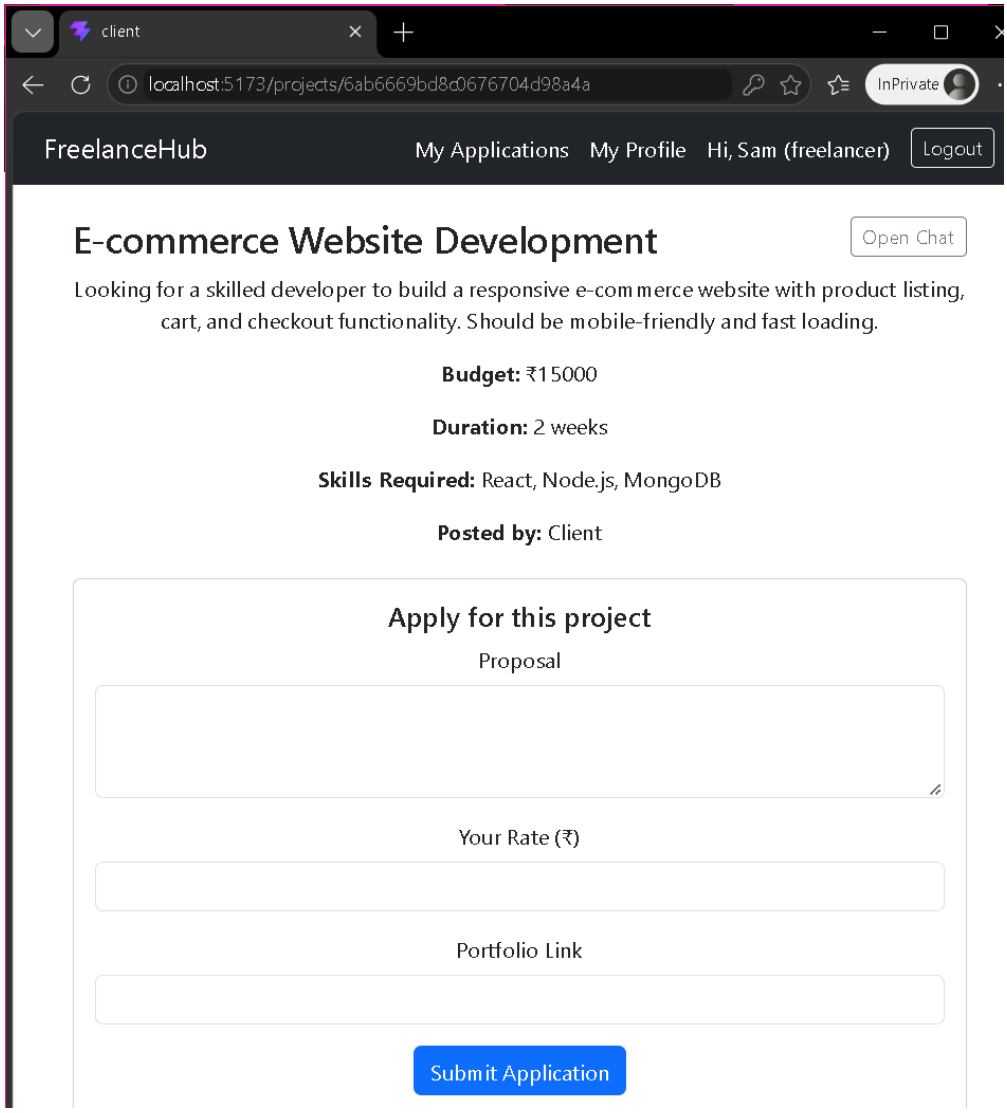


Figure: Apply to a Project

My Applications (Freelancer)

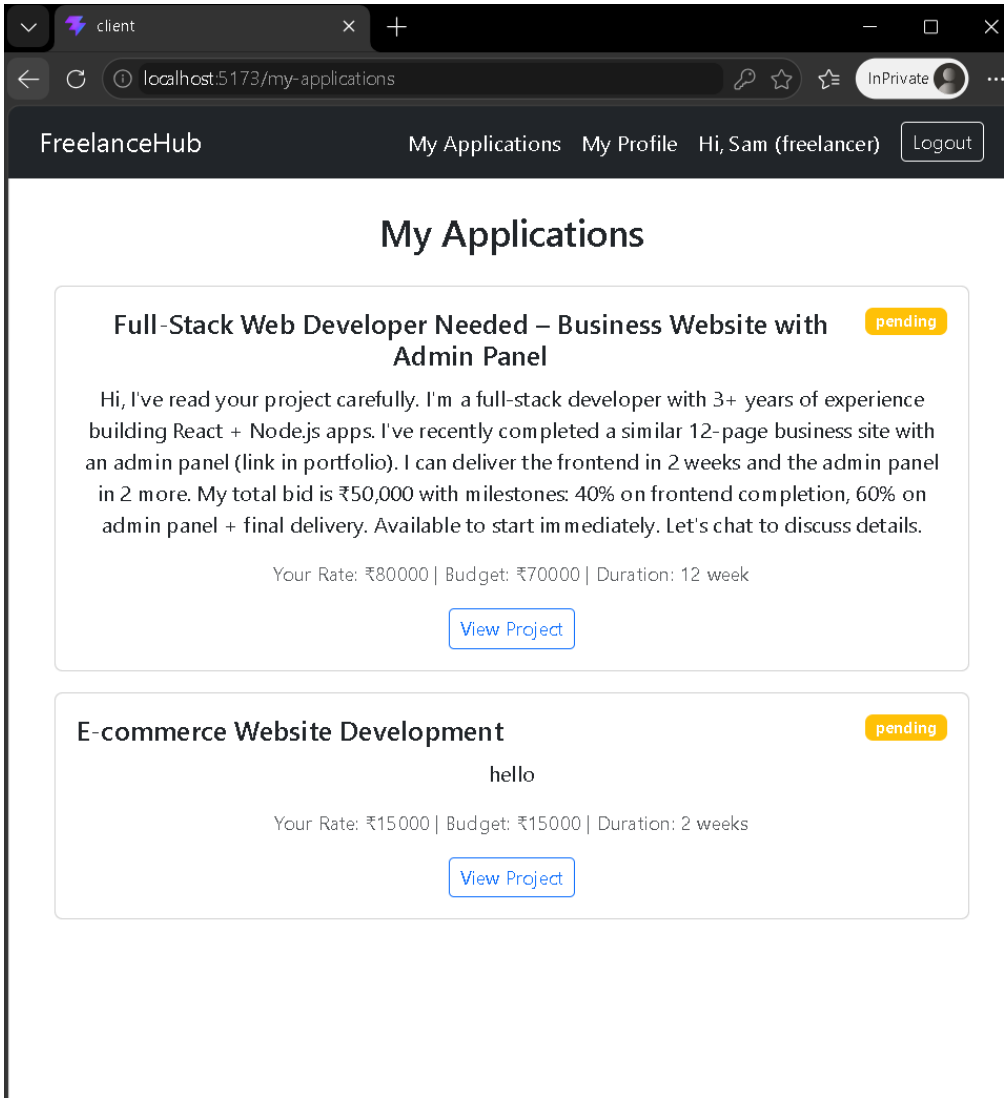


Figure: My Applications (Freelancer)

Real-time Chat

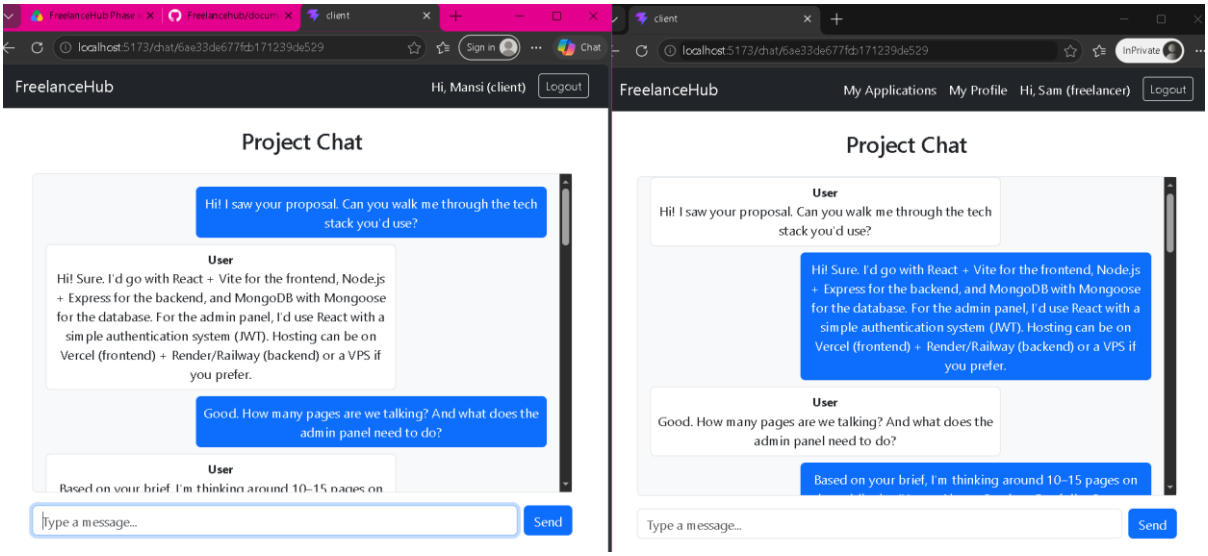


Figure: Real-time Chat

Demo Video & Code Links

Demo Video Link: FreelanceHub Phase wise implementation - Google Drive

Code Repository Link: https://github.com/mansiwagh2006/FreelanceHub-